

OPERATING SYSTEMS

Pintos: Report Project #5

Group #6

Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, May 6, 2024

1 FILES CHANGED

- `/pintos/threads/thread.h` (*modified*)
- `/pintos/threads/thread.c` (*modified*)
- `/pintos/userprog/process.c` (*modified*)
- `/pintos/userprog/syscall.c` (*modified*)

2 CHANGES

`/pintos/threads/thread.h`

- (*added*) **import** of `threads/synch.h`
- (*added*) **define** for `cur_status` field of `struct child_elem`
- (*added*) to **struct thread**:
 - **struct list child_list** list of all child threads of a thread
 - **struct semaphore child_load** when is < 0 , the current thread is blocked and waiting for its child to finish loading.
 - **struct semaphore child_exit** when is < 0 , the current thread is blocked and waiting for its child to terminate.
- (*added*) **struct child_elem** that is used to represent child threads in **struct list child_list**, it holds information about the status of the child (pid, if it's the first time its parent is waiting for it, current status, if it loaded successfully)
- (*added*) signature for `struct child_elem * thread_get_child(struct thread * parent, tid_t child_tid`

/pintos/devices/thread.c

- (added) **struct child_elem * thread_get_child(struct thread * parent, tid_t child_tid**, given a **struct thread *** to a parent thread and a **tid_t** of a child thread, returns the **struct child_elem *** if the parent contains that child_tid in its child_list or null if it doesn't.
- (modified) **tid_t thread_create(const char *name, int priority, thread_func *function, void *aux)**: We added the initialisation of the **struct child_elem *** by allocating the necessary memory and setting the default value of its fields. Then we push it into the **child_list** of the current thread.
- (modified) **static void init_thread(struct thread * t, const char * name, int priority)**: We initialised the parent to null, the semaphores with a 0 initial value and the child_list.

/pintos/userprog/process.c

- (modified) **static void start_process(void *file_name_)**: Increase the child_load semaphore of the child of the current thread because it means that the child (the current thread) has finished loading. Set the successful_load field of the child (which can be either true or false) to signal that the child was successfully loaded (or wasn't).
- (modified) **int process_wait(tid_t child_tid)**: Replaced the block/unblock with the semaphores: if the child process is alive, the parent will decrease the child_exit semaphore of its child, blocking itself while waiting for the child to finish execution.
- (modified) **void process_exit(void)**:
 - It prints the tests exit statements.
 - If the current thread has a parent (it is a child thread), it updates its data into the child_list of the parent (setting its current status to KILLED and exit status to -1). Removes its child_list and deallocates the memory, removes itself from its parent list, remove its own parent.

/pintos/userprog/syscall.c

- (added) **import** of userprog/pagedir.h
- (modified) added the handlers reference for wait() and exec().
- (added) **static void syscall_wait(struct intr_frame *f)**: handles the wait system call, calls **process_wait** on the pid popped from the stack.
- (added) **static void syscall_exec(struct intr_frame *f)**: pops the program pointer from the stack and checks if it's valid, if it's not valid, pushes -1 and calls syscall_exit. Checks if the current page dir of the given program is null, if it is, pushes -1 and calls syscall_exit. Now that we have checked that the given program is valid, call process_execute which will create a child and run it. Decreases the child_load semaphore of the newly created child. If the child was successfully loaded, returns the pid of the child.
- (modified) **static void syscall_exit(struct intr_frame *f)**: If the current thread has a parent (it is a child thread), it updates its data into the child_list of the parent (setting its current status to KILLED and exit status to EXITED).